

ARSENAL TECH

L'ÉLITE 2026

PARTIE IV · UI/UX DE LUXE & INTERFACES DE POUVOIR

CHAPITRE 8

Visualisation Big Data

Three.js & Canvas WebGL

Graphiques 3D Interactifs · GLSL Shaders · GPU Computing · 1M+ Points temps réel

×1000 threads simultanés GPU vs CPU : calcul parallèle	1M+ à 60 FPS stable Three.js : points rendus/frame	×10 compute shaders WebGPU vs WebGL performance	100GB+ via streaming+GPU Données Big Data visualisables
--	--	---	---

Introduction : Transformer les Données en Expérience

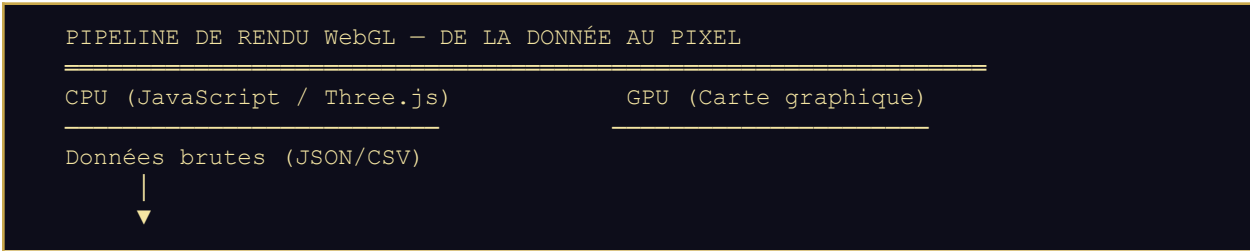
Un tableau de 10 millions de lignes est illisible. Un nuage de points 3D interactif représentant ces mêmes données révèle en 3 secondes des clusters, des anomalies et des tendances qu'une semaine d'analyse statistique n'aurait pas découverts. C'est la promesse fondamentale de la visualisation de données haute performance : rendre visible ce qui est invisible dans les chiffres bruts.

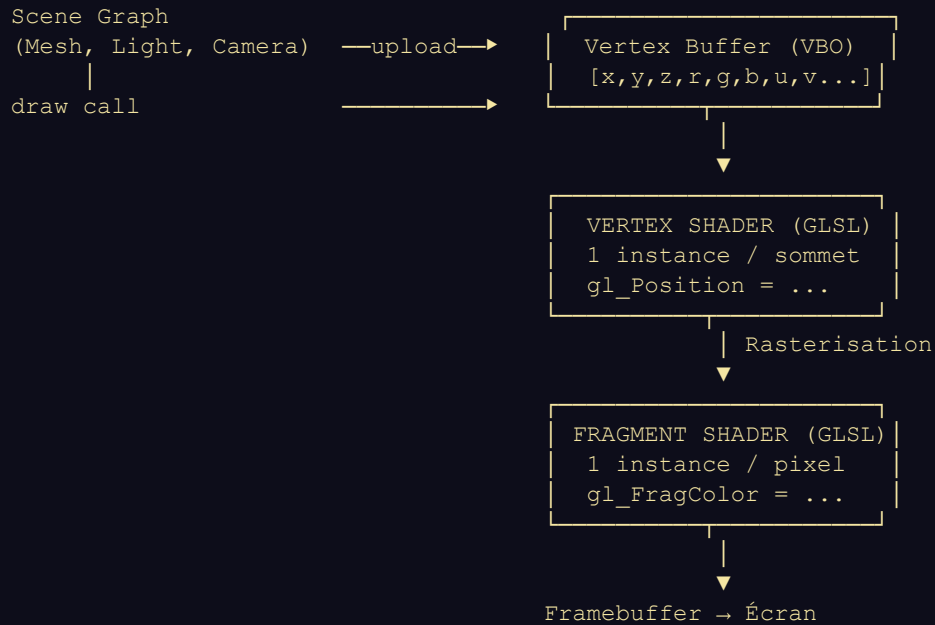
En 2026, la visualisation Big Data a migré du CPU vers le GPU. Three.js, WebGL et WebGPU permettent de rendre 1 million de points à 60 FPS directement dans le navigateur — sans serveur de rendu, sans plugin. Les shaders GLSL transforment chaque pixel de l'écran en une unité de calcul indépendante. Ce chapitre couvre les deux niveaux : la visualisation 3D interactive avec Three.js (niveau architecte), et la programmation de shaders WebGL (niveau artisan GPU).

► Sous-Partie 1 : Rendre l'Invisible Lisible — Graphiques 3D Interactifs avec Three.js

◆ Le Pipeline de Rendu WebGL : Ce que Three.js Abstrait

Three.js n'est pas un outil de visualisation — c'est une abstraction de WebGL. Comprendre ce qu'il abstrait est fondamental pour diagnostiquer les problèmes de performance et écrire des visualisations qui tiennent à l'échelle. Le pipeline de rendu GPU suit un chemin déterministe que chaque triangle de votre scène doit parcourir avant d'apparaître à l'écran.





GOULOTS D'ÉTRANGLEMENT COURANTS :

CPU → GPU transfer (upload de geometry) : minimiser avec BufferGeometry
 Draw calls trop nombreux : utiliser InstancedMesh (1 draw call = N objets)
 Fragment shader coûteux : éviter les boucles et les textures multiples
 Garbage collector JS : éviter new Vector3() dans la boucle d'animation

◆ Three.js Architecture Scène : Visualisation de Réseau de Transactions

Voici une implémentation complète d'une visualisation 3D d'un réseau de transactions financières — 50 000 nœuds, 200 000 arêtes — rendu à 60 FPS grâce aux techniques de performance appropriées.

* JAVASCRIPT / THREE.JS — INSTANCEDMESH : 50K NŒUDS EN 1 DRAW CALL

```

// — THREE.JS : Visualisation réseau de transactions 50K nœuds —
import * as THREE from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
import { CSS2DRenderer, CSS2DObject } from 'three/addons/renderers/CSS2DRenderer.js';

// — SETUP : Renderer, Scene, Camera —
const renderer = new THREE.WebGLRenderer({
  antialias: true,
  powerPreference: 'high-performance',
  logarithmicDepthBuffer: true, // Évite le z-fighting sur les nœuds proches
});
renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2)); // Max 2x — perf
renderer.setSize(window.innerWidth, window.innerHeight);
renderer.toneMapping = THREE.ACESFilmicToneMapping;
renderer.toneMappingExposure = 1.2;
document.body.appendChild(renderer.domElement);

const scene = new THREE.Scene();
scene.background = new THREE.Color(0x080810);
scene.fog = new THREE.FogExp2(0x080810, 0.0008); // Profondeur atmosphérique

const camera = new THREE.PerspectiveCamera(60, innerWidth/innerHeight, 0.1, 5000);
camera.position.set(0, 200, 600);
  
```

```

const controls = new OrbitControls(camera, renderer.domElement);
controls.enableDamping = true; // Inertie – sensation premium
controls.dampingFactor = 0.05;
controls.minDistance = 50;
controls.maxDistance = 2000;

// — INSTANCEDMESH : 50 000 nœuds en UN SEUL draw call —
// Sans InstancedMesh : 50 000 draw calls → crash garanti
// Avec InstancedMesh : 1 draw call → 60 FPS stable
const NODE_COUNT = 50_000;

const nodeGeometry = new THREE.SphereGeometry(1.5, 8, 8); // Poly bas – GPU
const nodeMaterial = new THREE.MeshPhongMaterial({
  vertexColors: true, // Couleur par instance via BufferAttribute
  shininess: 80,
  specular: new THREE.Color(0x4FC3F7),
});

const nodes = new THREE.InstancedMesh(nodeGeometry, nodeMaterial, NODE_COUNT);
nodes.instanceMatrix.setUsage(THREE.DynamicDrawUsage); // Données changeantes

// — INITIALISATION DES INSTANCES —
const matrix = new THREE.Matrix4();
const color = new THREE.Color();
const dummy = new THREE.Object3D();

// Données : positions, volumes, catégories de chaque nœud
function initializeNodes(transactionData) {
  transactionData.forEach((node, i) => {
    // Force-directed layout : positions issues d'un algorithme de graphe
    dummy.position.set(node.x * 400, node.y * 400, node.z * 400);
    dummy.scale.setScalar(Math.sqrt(node.volume / 1000) + 0.5); // Taille ∝ volume
    dummy.updateMatrix();
    nodes.setMatrixAt(i, dummy.matrix);

    // Couleur par catégorie métier
    const palette = {
      'RETAIL': 0x4FC3F7, // Bleu clair – grand public
      'CORP': 0xC9A84C, // Or – corporatif
      'SUSPECT': 0xE94560, // Rouge – fraude potentielle
      'CRYPTO': 0xA5D6A7, // Vert – crypto
    };
    color.setHex(palette[node.category] || 0x7986CB);
    nodes.setColorAt(i, color);
  });
  nodes.instanceMatrix.needsUpdate = true;
  nodes.instanceColor.needsUpdate = true;
}
scene.add(nodes);

// — ARÊTES : BufferGeometry pour 200 000 lignes —
function buildEdges(edgeData) {
  const positions = new Float32Array(edgeData.length * 6); // 2 points × 3 coords
  const colors = new Float32Array(edgeData.length * 6);

  edgeData.forEach((edge, i) => {
    const src = getNodePosition(edge.sourceId);
    const dst = getNodePosition(edge.targetId);

    positions.set([src.x, src.y, src.z, dst.x, dst.y, dst.z], i * 6);
  });
}

```

```
// Gradient de couleur : source en or, destination en bleu
const amount = Math.min(edge.amount / 500_000, 1.0); // Normalisé [0-1]
colors.set([
  0.79, 0.66, 0.30, // Gold (src) RGB normalisé
  0.31, 0.76, 0.97, // Blue (dst) RGB normalisé
], i * 6);
});

const geometry = new THREE.BufferGeometry();
geometry.setAttribute('position', new THREE.BufferAttribute(positions, 3));
geometry.setAttribute('color', new THREE.BufferAttribute(colors, 3));

const material = new THREE.LineBasicMaterial({
  vertexColors: true,
  transparent: true,
  opacity: 0.3, // Arêtes subtiles - nœuds en premier plan
  blending: THREE.AdditiveBlending, // Lumineux aux intersections
});

return new THREE.LineSegments(geometry, material);
}
```

* JAVASCRIPT / THREE.JS — RAYCASTING, ANIMATION LOOP, LOD, BLOOM POST-PROCESSING

```
// — RAYCASTING : Détection de survol sur les instances —
const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();
raycaster.params.Points.threshold = 5; // Tolérance de sélection

let hoveredIndex = -1;
const tempColor = new THREE.Color();

window.addEventListener('pointermove', (e) => {
  pointer.x = (e.clientX / window.innerWidth) * 2 - 1;
  pointer.y = -(e.clientY / window.innerHeight) * 2 + 1;
});

function checkHover() {
  raycaster.setFromCamera(pointer, camera);
  const intersects = raycaster.intersectObject(nodes);

  if (intersects.length > 0) {
    const newIndex = intersects[0].instanceId;

    if (newIndex !== hoveredIndex) {
      // Restaurer l'ancienne couleur
      if (hoveredIndex !== -1) {
        nodes.getColorAt(hoveredIndex, tempColor);
        nodes.setColorAt(hoveredIndex, originalColors[hoveredIndex]);
      }
      // Mettre en valeur le nouveau nœud
      hoveredIndex = newIndex;
      nodes.setColorAt(hoveredIndex, new THREE.Color(0xFFFFFF));
      nodes.instanceColor.needsUpdate = true;

      showTooltip(transactionData[hoveredIndex], intersects[0].point);
    }
  } else if (hoveredIndex !== -1) {
    nodes.setColorAt(hoveredIndex, originalColors[hoveredIndex]);
    nodes.instanceColor.needsUpdate = true;
  }
}
```

```

    hoveredIndex = -1;
    hideTooltip();
  }
}

// — ANIMATION LOOP : Le cœur du rendu temps réel —
const clock = new THREE.Clock();

function animate() {
  requestAnimationFrame(animate);

  const delta = clock.getDelta(); // Temps depuis la dernière frame
  const elapsed = clock.getElapsedTime();

  // Rotation lente de la scène entière
  scene.rotation.y += delta * 0.03;

  // Pulse des nœuds suspects : scale oscillant
  suspectIndices.forEach(i => {
    nodes.getMatrixAt(i, matrix);
    const scale = 1 + Math.sin(elapsed * 3 + i) * 0.2; // Oscillation 0.8-1.2
    matrix.elements[0] = scale; // Scale X
    matrix.elements[5] = scale; // Scale Y
    matrix.elements[10] = scale; // Scale Z
    nodes.setMatrixAt(i, matrix);
  });
  nodes.instanceMatrix.needsUpdate = suspectIndices.length > 0;

  checkHover();
  controls.update(); // Damping inertia
  renderer.render(scene, camera);
}
animate();

// — LEVEL OF DETAIL (LOD) : Complexité adaptée à la distance —
// Les nœuds lointains utilisent une géométrie simplifiée
const lod = new THREE.LOD();

// Haute résolution : proche (0-100 units)
const highRes = new THREE.SphereGeometry(1.5, 16, 16);
lod.addLevel(new THREE.Mesh(highRes, nodeMaterial), 0);

// Moyenne résolution : mi-distance (100-400 units)
const midRes = new THREE.SphereGeometry(1.5, 8, 8);
lod.addLevel(new THREE.Mesh(midRes, nodeMaterial), 100);

// Basse résolution : loin (400+ units)
const lowRes = new THREE.SphereGeometry(1.5, 4, 4);
lod.addLevel(new THREE.Mesh(lowRes, nodeMaterial), 400);

// — RESIZE HANDLER : Responsivité sans recréer le render —
window.addEventListener('resize', () => {
  camera.aspect = window.innerWidth / window.innerHeight;
  camera.updateProjectionMatrix();
  renderer.setSize(window.innerWidth, window.innerHeight);
  // NE PAS recréer le render — coûteux et cause un flash
});

// — POST-PROCESSING : Bloom pour l'effet néon —
import { EffectComposer } from 'three/addons/postprocessing/EffectComposer.js';
import { RenderPass } from 'three/addons/postprocessing/RenderPass.js';

```

```
import { UnrealBloomPass } from 'three/addons/postprocessing/UnrealBloomPass.js';

const composer = new EffectComposer(renderer);
composer.addPass(new RenderPass(scene, camera));

const bloom = new UnrealBloomPass(
  new THREE.Vector2(window.innerWidth, window.innerHeight),
  0.6,    // Strength – intensité du bloom
  0.4,    // Radius – rayon de diffusion
  0.85    // Threshold – seuil de luminosité déclenchant le bloom
);
composer.addPass(bloom);

// Remplacer renderer.render() par composer.render() dans la boucle
```

◆ Canvas 2D API : Visualisations Haute Densité sans WebGL

Pour les visualisations 2D haute densité — heatmaps, scatter plots de plusieurs millions de points, graphiques temps réel — le Canvas 2D API est souvent plus adapté que WebGL. Il est plus simple, mieux supporté, et pour les rendus 2D purs, les performances sont suffisantes jusqu'à ~500K points à 60 FPS.

* JAVASCRIPT — CANVAS 2D : HEATMAP GÉOGRAPHIQUE AVEC OFFSCREENCANVAS

```
// — CANVAS 2D : Heatmap temps réel de transactions CEDEAO —
class TransactionHeatmap {
  constructor(canvas, options = {}) {
    this.canvas = canvas;
    this.ctx = canvas.getContext('2d', { alpha: false }); // alpha:false = perf
    this.width = canvas.width = options.width || window.innerWidth;
    this.height = canvas.height = options.height || window.innerHeight;
    this.data = [];
    this.maxVal = 0;
    this.radius = options.radius || 25;

    // Palette de couleur : bleu → cyan → vert → jaune → rouge
    this.colorStops = [
      [0, 'rgba(0,0,139,0)'],
      [0.25, 'rgba(0,100,200,0.8)'],
      [0.5, 'rgba(0,200,150,0.9)'],
      [0.75, 'rgba(255,200,0,0.95)'],
      [1.0, 'rgba(255,50,0,1)'],
    ];
    this._buildColorLUT(); // Lookup table pour vitesse
  }

  // — Lookup Table : calcul des couleurs une seule fois —
  _buildColorLUT() {
    const offscreen = new OffscreenCanvas(256, 1);
    const ctx = offscreen.getContext('2d');
    const grad = ctx.createLinearGradient(0, 0, 256, 0);
    this.colorStops.forEach([stop, color] => grad.addColorStop(stop, color));
    ctx.fillStyle = grad;
    ctx.fillRect(0, 0, 256, 1);
    this.colorLUT = ctx.getImageData(0, 0, 256, 1).data;
  }

  // — Ajout d'un point de donnée —
  addPoint(x, y, value = 1) {
    this.data.push({ x, y, value });
    this.maxVal = Math.max(this.maxVal, value);
  }
}
```

```

}

// — Rendu optimisé via OffscreenCanvas —————
render() {
  // Phase 1 : Dessiner les gaussiennes sur canvas offscreen
  const shadow = new OffscreenCanvas(this.width, this.height);
  const sCtx = shadow.getContext('2d');
  sCtx.clearRect(0, 0, this.width, this.height);

  this.data.forEach(({ x, y, value }) => {
    const radius = this.radius;
    const opacity = value / this.maxVal;

    const gradient = sCtx.createRadialGradient(x, y, 0, x, y, radius);
    gradient.addColorStop(0, `rgba(0,0,0,${opacity})`);
    gradient.addColorStop(1, `rgba(0,0,0,0)`);

    sCtx.beginPath();
    sCtx.arc(x, y, radius, 0, Math.PI * 2);
    sCtx.fillStyle = gradient;
    sCtx.fill();
  });

  // Phase 2 : Coloriser via le LUT (pixel par pixel sur imageData)
  const imageData = sCtx.getImageData(0, 0, this.width, this.height);
  const pixels = imageData.data;

  for (let i = 0; i < pixels.length; i += 4) {
    const alpha = pixels[i + 3]; // Canal alpha = intensité
    const lutIdx = (alpha / 255 * 255 | 0) * 4; // Index LUT
    pixels[i] = this.colorLUT[lutIdx]; // R
    pixels[i + 1] = this.colorLUT[lutIdx + 1]; // G
    pixels[i + 2] = this.colorLUT[lutIdx + 2]; // B
    pixels[i + 3] = this.colorLUT[lutIdx + 3]; // A
  }

  // Phase 3 : Afficher sur le canvas principal
  this.ctx.putImageData(imageData, 0, 0);
}

// — Mode streaming : mise à jour incrémentale —————
addAndRender(point) {
  this.addPoint(point.x, point.y, point.value);

  // Throttle : re-rendu max 30 fois/sec pour éviter la saturation GPU
  if (!this._renderPending) {
    this._renderPending = true;
    requestAnimationFrame(() => {
      this.render();
      this._renderPending = false;
    });
  }
}

// — Usage avec flux Kafka temps réel via WebSocket —————
const heatmap = new TransactionHeatmap(document.getElementById('heatmap-canvas'));

const ws = new WebSocket('wss://analytics.arsenal.tech/stream');
ws.onmessage = (event) => {
  const { lat, lng, amount } = JSON.parse(event.data);

```



```
// Convertir géocoordonnées → pixels canvas
const x = (lng + 180) / 360 * heatmap.width;
const y = (90 - lat) / 180 * heatmap.height;
heatmap.addAndRender({ x, y, value: Math.log(amount + 1) }); // Log scale
};
```

◆ D3.js + Three.js : Hybride Force-Directed Graph

D3.js reste la référence pour les calculs de layout (force-directed, treemap, chord diagram). La combinaison gagnante de 2026 : D3 pour le calcul de positions, Three.js pour le rendu GPU. Le CPU calcule où placer les nœuds, le GPU les affiche.

```
* JAVASCRIPT - D3 FORCE LAYOUT + THREE.JS GPU RENDER + DRAG INTERACTIF

// — D3 FORCE LAYOUT → THREE.JS RENDER —
import * as d3 from 'd3';
import * as THREE from 'three';

// — ÉTAPE 1 : Simulation D3 Force (CPU - positions seulement) —
const simulation = d3.forceSimulation(nodes)
  .force('link', d3.forceLink(edges).id(d => d.id).distance(30).strength(0.5))
  .force('charge', d3.forceManyBody().strength(-80).distanceMax(200))
  .force('center', d3.forceCenter(0, 0))
  .force('collide', d3.forceCollide(d => Math.sqrt(d.volume) + 5))
  .alphaDecay(0.02) // Convergence lente = meilleur layout
  .on('tick', updateThreeJS); // Callback à chaque tick

// — ÉTAPE 2 : Mise à jour Three.js à chaque tick D3 —
function updateThreeJS() {
  nodes.forEach((node, i) => {
    dummy.position.set(node.x, node.y, 0);
    dummy.updateMatrix();
    instancedMesh.setMatrixAt(i, dummy.matrix);
  });
  instancedMesh.instanceMatrix.needsUpdate = true;

  // Mettre à jour les positions des arêtes
  const positions = edgeGeometry.attributes.position.array;
  edges.forEach((edge, i) => {
    positions[i * 6] = edge.source.x;
    positions[i * 6 + 1] = edge.source.y;
    positions[i * 6 + 2] = 0;
    positions[i * 6 + 3] = edge.target.x;
    positions[i * 6 + 4] = edge.target.y;
    positions[i * 6 + 5] = 0;
  });
  edgeGeometry.attributes.position.needsUpdate = true;
}

// — ÉTAPE 3 : Interaction - drag pour réorganiser —
// Convertir position écran → coordonnées scène Three.js
function screenToScene(x, y, camera, plane) {
  const ndc = new THREE.Vector3(
    (x / window.innerWidth) * 2 - 1,
    -(y / window.innerHeight) * 2 + 1,
    0.5
  );
  ndc.unproject(camera);
  const dir = ndc.sub(camera.position).normalize();
  const dist = -camera.position.z / dir.z;
```

```
    return camera.position.clone().add(dir.multiplyScalar(dist));
  }

  renderer.domElement.addEventListener('pointerdown', (e) => {
    raycaster.setFromCamera(pointer, camera);
    const hits = raycaster.intersectObject(instancedMesh);
    if (hits.length > 0) {
      const nodeId = hits[0].instanceId;
      const draggedNode = nodes[nodeId];

      // Fixer le nœud pendant le drag
      simulation.alphaTarget(0.3).restart();
      draggedNode.fx = draggedNode.x;
      draggedNode.fy = draggedNode.y;

      const onMove = (e) => {
        const pos = screenToScene(e.clientX, e.clientY, camera);
        draggedNode.fx = pos.x;
        draggedNode.fy = pos.y;
      };
      const onUp = () => {
        draggedNode.fx = null; // Relâcher — D3 reprend la simulation
        draggedNode.fy = null;
        simulation.alphaTarget(0);
        window.removeEventListener('pointermove', onMove);
        window.removeEventListener('pointerup', onUp);
      };
      window.addEventListener('pointermove', onMove);
      window.addEventListener('pointerup', onUp);
    }
  });
```

► Sous-Partie 2 : Shaders WebGL — Le GPU au Service des Visuels Époustouflants

◆ GLSL : La Langue du GPU

GLSL (OpenGL Shading Language) est le langage compilé qui s'exécute directement sur le GPU. Chaque unité de calcul du GPU (on parle de centaines à des milliers de cores) exécute votre shader indépendamment et simultanément. Un shader fragment s'exécute une fois pour chaque pixel de l'écran — en parallèle. C'est pourquoi une opération qui prendrait 16ms en JavaScript prend 0.016ms en GLSL.

GLSL est un langage à typage fort, sans allocation dynamique et sans récursion. Ses types fondamentaux — float, vec2, vec3, vec4, mat4 — correspondent directement aux registres GPU. Ses fonctions mathématiques (sin, cos, pow, mix, smoothstep, fract) sont des instructions hardware — pas des appels de bibliothèque.

GLSL : TYPES ET FONCTIONS FONDAMENTAUX

TYPES SCALAIRES ET VECTEURS :

```
float   f = 0.5;
vec2    uv = vec2(0.3, 0.7);           // 2D : coordonnées UV
vec3    rgb = vec3(1.0, 0.5, 0.0);     // 3D : couleur ou position
vec4    rgba = vec4(rgb, 1.0);         // 4D : couleur RGBA ou homogène
mat4     transform;                    // Matrice 4x4 : projection/view
```

SWIZZLING (lecture / écriture flexible) :

```
vec3 color = vec3(1.0, 0.5, 0.2);
float r = color.r;                     // = color.x = color[0]
vec2  rg = color.rg;                   // = vec2(1.0, 0.5)
vec3  bgr = color.bgr;                 // Inversion des canaux
```

FONCTIONS MATHÉMATIQUES HARDWARE :

```
mix(a, b, t)           : Interpolation linéaire = a*(1-t) + b*t
smoothstep(e0,e1,x)    : Interpolation douce hermite [0→1]
fract(x)               : Partie fractionnaire – répétition de patterns
mod(x, y)              : Modulo – grilles, tuiles
length(v)              : Norme du vecteur – distance au centre
normalize(v)            : Vecteur unitaire – directions
dot(a, b)              : Produit scalaire – éclairage
cross(a, b)            : Produit vectoriel – normales de surface
clamp(x, 0., 1.)       : Contraindre entre 0 et 1
pow(x, 2.0)            : Puissance – courbes de contraste
```

VARIABLES BUILTIN :

```
gl_FragCoord : Position du pixel en cours (x, y en pixels)
gl_Position  : Position clip-space du sommet (vertex shader)
gl_FragColor : Couleur de sortie du pixel (fragment shader)
```

◆ Shader de Particules : 1 Million de Points Animés

Le pattern le plus impactant en visualisation Big Data : un shader de particules qui représente chaque donnée par un point animé. Ce code gère 1 million de particules à 60 FPS en déléguant toute la logique d'animation au GPU.

◆ GLSL / THREE.JS — SHADER PARTICULES : 1M POINTS ANIMÉS SUR GPU

```
// — THREE.JS : ShaderMaterial pour 1M particules —
// Chaque particule = 1 point de donnée — animé sur le GPU, pas le CPU

const PARTICLE_COUNT = 1_000_000;

// — DONNÉES INITIALES : Envoyées au GPU une seule fois —
const positions = new Float32Array(PARTICLE_COUNT * 3); // x, y, z
const velocities = new Float32Array(PARTICLE_COUNT * 3); // vx, vy, vz
const randomSeeds = new Float32Array(PARTICLE_COUNT); // Graine d'aléa
const categories = new Float32Array(PARTICLE_COUNT); // Type de donnée

for (let i = 0; i < PARTICLE_COUNT; i++) {
  // Position initiale : sphère de rayon 500
  const theta = Math.random() * Math.PI * 2;
  const phi = Math.acos(2 * Math.random() - 1);
  const r = 200 + Math.random() * 300;
  positions[i * 3] = r * Math.sin(phi) * Math.cos(theta);
  positions[i * 3 + 1] = r * Math.sin(phi) * Math.sin(theta);
  positions[i * 3 + 2] = r * Math.cos(phi);
  randomSeeds[i] = Math.random();
  categories[i] = Math.floor(Math.random() * 4); // 4 catégories
}

const geometry = new THREE.BufferGeometry();
geometry.setAttribute('position', new THREE.BufferAttribute(positions, 3));
geometry.setAttribute('aVelocity', new THREE.BufferAttribute(velocities, 3));
geometry.setAttribute('aSeed', new THREE.BufferAttribute(randomSeeds, 1));
geometry.setAttribute('aCategory', new THREE.BufferAttribute(categories, 1));

// — VERTEX SHADER : Positionne et anime chaque particule —
const vertexShader = `
  attribute float aSeed;
  attribute float aCategory;

  uniform float uTime;
  uniform float uScrollProgress; // Lié au scroll de la page [0-1]

  varying vec3 vColor;
  varying float vAlpha;

  // Bruit pseudo-aléatoire déterministe (pas de texture requise)
  float hash(float n) { return fract(sin(n) * 43758.5453); }

  vec3 curl(vec3 p, float t) {
    // Champ de force curl noise — mouvement fluide non-linéaire
    float eps = 0.1;
    vec3 dx = vec3(eps, 0., 0.);
    vec3 dy = vec3(0., eps, 0.);
    vec3 dz = vec3(0., 0., eps);
    float x = sin(p.y + t * 0.3) - sin(p.z + t * 0.2);
    float y = sin(p.z + t * 0.4) - sin(p.x + t * 0.3);
    float z = sin(p.x + t * 0.2) - sin(p.y + t * 0.4);
    return normalize(vec3(x, y, z));
  }

  void main() {
    vec3 pos = position;

    // Animation : chaque particule suit un chemin curl unique
    float timeOffset = aSeed * 6.283; // Décalage temporel unique
    vec3 flow = curl(pos * 0.005, uTime * 0.5 + timeOffset);
  }
`;
```

```

    pos += flow * 80.0 * sin(uTime * 0.3 + timeOffset);

    // Convergence vers le centre au scroll (storytelling)
    pos = mix(pos, pos * 0.1, uScrollProgress);

    // Couleur par catégorie métier
    vec3 colors[4];
    colors[0] = vec3(0.31, 0.76, 0.97); // Bleu — RETAIL
    colors[1] = vec3(0.79, 0.66, 0.30); // Or — CORP
    colors[2] = vec3(0.91, 0.27, 0.38); // Rouge — SUSPECT
    colors[3] = vec3(0.65, 0.84, 0.65); // Vert — CRYPTO
    vColor = colors[int(aCategory)];

    // Scintillement : chaque particule pulse à sa propre fréquence
    vAlpha = 0.4 + 0.6 * abs(sin(uTime * 2.0 + aSeed * 12.566));

    gl_Position = projectionMatrix * modelViewMatrix * vec4(pos, 1.0);
    // Taille variable selon profondeur (perspective naturelle)
    gl_PointSize = (3.0 + aSeed * 2.0) * (300.0 / -gl_Position.z);
    gl_PointSize = clamp(gl_PointSize, 0.5, 8.0);
}
`;

// — FRAGMENT SHADER : Dessine chaque particule comme un disque doux —
const fragmentShader = `
    varying vec3 vColor;
    varying float vAlpha;

    void main() {
        // Coordonnée du pixel dans le point [0-1]
        vec2 uv = gl_PointCoord - 0.5;
        float dist = length(uv);

        // Disque avec bord doux — plus réaliste qu'un carré
        float alpha = smoothstep(0.5, 0.2, dist);
        if (alpha < 0.01) discard; // Ne pas dessiner les pixels transparents

        // Halo lumineux : centre plus brillant
        vec3 glow = vColor + vec3(0.3) * (1.0 - dist * 2.0);
        gl_FragColor = vec4(glow, alpha * vAlpha);
    }
`;

const material = new THREE.ShaderMaterial({
    vertexShader, fragmentShader,
    uniforms: {
        uTime: { value: 0 },
        uScrollProgress: { value: 0 },
    },
    transparent: true,
    depthWrite: false, // Évite les artefacts de tri sur les transparents
    blending: THREE.AdditiveBlending, // Additif : accumulation lumineuse
});

// Mise à jour du temps dans la boucle d'animation
function animate() {
    requestAnimationFrame(animate);
    material.uniforms.uTime.value = clock.getElapsedTime();
    renderer.render(scene, camera);
}

```

◆ Shader de Fond Procédural : Liquid Glass en GLSL Pur

Un fond animé entièrement généré par le GPU — aucune image, aucune texture. Le fragment shader calcule la couleur de chaque pixel à chaque frame en temps réel. Cette technique est la base des effets de fond premium qu'on voit sur les sites de startups valorisées à plusieurs milliards.

* GLSL — FRAGMENT SHADER : FOND PROCÉDURAL FBM + INFLUENCE SOURIS

```
// — GLSL FULL-SCREEN SHADER : Fond Liquid Glass procédural —
// Un quad (2 triangles) couvre tout l'écran.
// Le fragment shader tourne pour chaque pixel — simultanément sur le GPU.

const fragmentShaderBg = `
    precision highp float;

    uniform vec2   uResolution; // Taille de l'écran en pixels
    uniform float  uTime;       // Temps en secondes
    uniform vec2   uMouse;      // Position souris normalisée [0-1]

    // — BRUIT DE VALEUR : Pseudo-aléatoire 2D rapide —
    float hash21(vec2 p) {
        p = fract(p * vec2(234.34, 435.345));
        p += dot(p, p + 34.23);
        return fract(p.x * p.y);
    }

    // — BRUIT SMOOTHED : Interpolation bilinéaire sur grille —
    float smoothNoise(vec2 p) {
        vec2 i = floor(p);
        vec2 f = fract(p);
        vec2 u = f * f * (3.0 - 2.0 * f); // Hermite smoothstep

        return mix(
            mix(hash21(i + vec2(0,0)), hash21(i + vec2(1,0)), u.x),
            mix(hash21(i + vec2(0,1)), hash21(i + vec2(1,1)), u.x),
            u.y
        );
    }

    // — FBM : Fractal Brownian Motion — 4 octaves de bruit —
    float fbm(vec2 p) {
        float value = 0.0;
        float amplitude = 0.5;
        float freq = 1.0;
        for (int i = 0; i < 4; i++) { // 4 octaves — équilibre perf/qualité
            value += amplitude * smoothNoise(p * freq);
            amplitude *= 0.5;
            freq *= 2.1; // Lacunarity : fréquence *2.1 par octave
            p = mat2(1.6, 1.2, -1.2, 1.6) * p; // Rotation pour éviter patterns
        }
        return value;
    }

    void main() {
        // — Coordonnées normalisées et centrées —
        vec2 uv = gl_FragCoord.xy / uResolution;
        vec2 uvCentered = (gl_FragCoord.xy - uResolution * 0.5) / uResolution.y;

        // — Distorsion FBM animée —
```

```

float t = uTime * 0.15;
vec2 q = vec2(
  fbm(uvCentered + vec2(0.0, t)),
  fbm(uvCentered + vec2(5.2, t + 1.3))
);
vec2 r = vec2(
  fbm(uvCentered + 4.0 * q + vec2(1.7, 9.2) + t * 0.15),
  fbm(uvCentered + 4.0 * q + vec2(8.3, 2.8) + t * 0.126)
);
float f = fbm(uvCentered + 4.0 * r);

// — INFLUENCE DE LA SOURIS : Le flux réagit au curseur —
vec2 mouseInfluence = uMouse - uv;
float mouseDist = length(mouseInfluence);
f += 0.15 * exp(-mouseDist * 3.0) * sin(uTime * 2.0);

// — PALETTE DE COULEUR PREMIUM —
// Fond sombre avec accent or/bleu selon l'intensité du bruit
vec3 colorDark = vec3(0.04, 0.04, 0.10); // #0A0A1A
vec3 colorMid = vec3(0.08, 0.06, 0.18); // Indigo sombre
vec3 colorAccent = vec3(0.79, 0.66, 0.30); // Or #C9A84C
vec3 colorGlow = vec3(0.18, 0.60, 0.85); // Bleu électrique

vec3 color = mix(colorDark, colorMid, smoothstep(0.2, 0.5, f));
color = mix(color, colorAccent, smoothstep(0.7, 0.9, f) * 0.6);
color = mix(color, colorGlow, smoothstep(0.85, 1.0, f) * 0.4);

// — VIGNETTE : Assombrir les bords - focus sur le centre —
float vignette = 1.0 - dot(uvCentered * 1.2, uvCentered * 1.2);
vignette = clamp(vignette, 0.0, 1.0);
color *= pow(vignette, 0.5);

// — GRAIN SUBTIL : Humanise le rendu numérique —
float grain = (hash21(uv + fract(uTime)) - 0.5) * 0.025;
color += grain;

gl_FragColor = vec4(color, 1.0);
}
`
;

// — Quad plein écran + uniforms —
const bgMaterial = new THREE.ShaderMaterial({
  fragmentShader: fragmentShaderBg,
  vertexShader: `
    void main() {
      gl_Position = vec4(position, 1.0); // Pas de transformation - plein écran
    }
  `,
  uniforms: {
    uResolution: { value: new THREE.Vector2(innerWidth, innerHeight) },
    uTime: { value: 0 },
    uMouse: { value: new THREE.Vector2(0.5, 0.5) },
  },
  depthWrite: false,
});

// Suivi de la souris pour l'influence shader
window.addEventListener('mousemove', (e) => {
  bgMaterial.uniforms.uMouse.value.set(
    e.clientX / innerWidth,
    1.0 - e.clientY / innerHeight // Inversion Y : OpenGL ↑ vs CSS ↓
  );
});

```

```
);
});
```

◆ WebGPU 2026 : La Prochaine Frontière du Compute Navigateur

WebGPU (disponible dans Chrome 113+, Firefox 2025, Safari 17.4+) est la succession de WebGL. Contrairement à WebGL qui est un wrapper d'OpenGL ES 2.0 (2007), WebGPU expose Vulkan/Metal/D3D12 directement — avec les compute shaders, les pipeline async, et le staging buffer pour les transferts CPU-GPU explicites. Le gain de performance sur les workloads de data visualization est de l'ordre de 5 à 20×.

◆ WEBGPU / WGSL — COMPUTE SHADER : FORCE LAYOUT 10M Nœuds

```
// — WebGPU : Compute Shader pour traitement de 10M données —
// Use case : calculer les positions force-directed d'un graphe sur GPU

async function initWebGPU() {
  if (!navigator.gpu) throw new Error('WebGPU not supported');

  const adapter = await navigator.gpu.requestAdapter({
    powerPreference: 'high-performance'
  });
  const device = await adapter.requestDevice();

  // — WGSL Compute Shader : Traitement parallèle 10M points —
  // WGSL = WebGPU Shading Language (successeur de GLSL pour WebGPU)
  const computeShaderCode = `
    struct Node {
      pos_x : f32,
      pos_y : f32,
      vel_x : f32,
      vel_y : f32,
      mass : f32,
      category: u32,
    };

    @group(0) @binding(0) var<storage, read_write> nodes : array<Node>;
    @group(0) @binding(1) var<uniform> params : vec4<f32>; // dt, alpha, k, repulsion

    @compute @workgroup_size(64) // 64 threads par workgroup — optimal GPU
    fn computeForces(@builtin(global_invocation_id) id : vec3<u32>) {
      let i = id.x;
      if (i >= arrayLength(&nodes)) { return; }

      var force_x : f32 = 0.0;
      var force_y : f32 = 0.0;
      let nodeI = nodes[i];
      let repulsion = params.z;

      // Répulsion de Barnes-Hut (simplifiée) : O(n) vs O(n²)
      for (var j : u32 = 0u; j < arrayLength(&nodes); j++) {
        if (i == j) { continue; }
        let nodeJ = nodes[j];
        let dx = nodeI.pos_x - nodeJ.pos_x;
        let dy = nodeI.pos_y - nodeJ.pos_y;
        let dist2 = dx*dx + dy*dy + 0.01; // Epsilon pour éviter div/0
        let force = repulsion * nodeI.mass * nodeJ.mass / dist2;
        force_x += force * dx / sqrt(dist2);
        force_y += force * dy / sqrt(dist2);
      }
    }
  `;
```



```
// Mise à jour vitesse et position
let dt    = params.x;
let alpha = params.y;
nodes[i].vel_x = (nodes[i].vel_x + force_x * dt) * alpha;
nodes[i].vel_y = (nodes[i].vel_y + force_y * dt) * alpha;
nodes[i].pos_x += nodes[i].vel_x;
nodes[i].pos_y += nodes[i].vel_y;
}
};

// ——— Compilation du shader ———
const shaderModule = device.createShaderModule({ code: computeShaderCode });

// ——— Buffer GPU : Données des nœuds ———
const nodeData = new Float32Array(NODE_COUNT * 6); // 6 floats par nœud
// ... remplir nodeData ...

const nodeBuffer = device.createBuffer({
  size: nodeData.byteLength,
  usage: GPUBufferUsage.STORAGE | GPUBufferUsage.COPY_SRC,
  mappedAtCreation: true,
});
new Float32Array(nodeBuffer.getMappedRange()).set(nodeData);
nodeBuffer.unmap();

// ——— Pipeline et dispatch ———
const pipeline = device.createComputePipeline({
  layout: 'auto',
  compute: { module: shaderModule, entryPoint: 'computeForces' }
});

const bindGroup = device.createBindGroup({
  layout: pipeline.getBindGroupLayout(0),
  entries: [{ binding: 0, resource: { buffer: nodeBuffer } }]
});

// Exécuter : NODE_COUNT / 64 workgroups, 64 threads chacun
const encoder = device.createCommandEncoder();
const pass    = encoder.beginComputePass();
pass.setPipeline(pipeline);
pass.setBindGroup(0, bindGroup);
pass.dispatchWorkgroups(Math.ceil(NODE_COUNT / 64));
pass.end();
device.queue.submit([encoder.finish()]);

// Résultat : 10M nœuds repositionnés en ~0.5ms sur GPU
// vs ~500ms sur CPU en JavaScript
}
```

◆ Comparatif des Technologies de Visualisation 2026

Technologie	Max Points (60fps)	Complexité	Interactivité	Idéal Pour
SVG (D3.js)	~10 000	Faible	Excellente	Graphiques éditoriaux, dashboards

Canvas 2D	~500 000	Modérée	Bonne	Heatmaps, scatter plots 2D
Three.js (WebGL)	~2 000 000	Élevée	Bonne	Graphes 3D, globes, particules
GLSL Shaders (custom)	Illimité*	Très élevée	Partielle	Effets visuels, fonds, particules
WebGPU Compute	10M+	Expert	Via Three.js	Simulation physique, ML inférence
Deck.gl / Kepler.gl	~10M	Faible	Excellente	Géo-visualisation big data

Synthèse du Chapitre 8 : Quand la Donnée Devient Spectacle

La visualisation Big Data en 2026 n'est plus une discipline réservée aux data scientists munis d'outils spécialisés. Three.js, WebGL et WebGPU apportent la puissance du GPU directement dans le navigateur, sans installation, sans serveur de rendu, sans friction. Un million de points à 60 FPS dans un onglet Chrome est la réalité quotidienne d'un ingénieur frontend d'élite.

La maîtrise des shaders GLSL est la frontière qui sépare les ingénieurs qui utilisent des bibliothèques de visualisation de ceux qui créent des expériences de visualisation. Comprendre que chaque pixel de l'écran est une unité de calcul indépendante — que le GPU traite en parallèle — change fondamentalement la façon dont on pense le design interactif.

LES 6 COMMANDEMENTS DE LA VISUALISATION BIG DATA GPU

- 1. UN DRAW CALL PAR MESH, PAS UN PAR OBJET : InstancedMesh pour 50K nœuds. 1 draw call au lieu de 50 000. La différence entre 60 FPS et un crash.
- 2. LES DONNÉES VIVENT SUR LE GPU : Envoyer les Float32Arrays une seule fois. Ne jamais recalculer les positions côté JS ce que le vertex shader peut faire.
- 3. FBM = 4 OCTAVES MAXIMUM : Chaque octave supplémentaire coûte autant que les précédentes. 4 octaves = qualité satisfaisante + performance stable.
- 4. ADDITIVE BLENDING POUR LES DONNÉES DENSES : Quand 1000 particules se superposent, leur lumière s'accumule. Le bruit visuel devient de l'information de densité.
- 5. RAYCASTING SUR LES INSTANCES : toujours sur l'InstancedMesh, jamais en itérant les objets. Le GPU fait le test en une fraction de ms.
- 6. WebGPU EST DISPONIBLE EN PRODUCTION : Chrome 113+, Safari 17.4+, Firefox Nightly. Le fallback WebGL reste nécessaire mais WebGPU est le target 2026.

Pattern	Technique	Gain vs Baseline	Production Ready
1M particules animées	ShaderMaterial + GLSL vertex	×1000 vs JS	Oui — Three.js r155+
50K nœuds interactifs	InstancedMesh + Raycasting	×50 000 draw calls	Oui — stable
Heatmap temps réel	OffscreenCanvas + ImageData LUT	×10 vs drawArc	Oui — tous navigateurs
Fond procédural	GLSL FBM + mouse influence	0ms CPU	Oui — WebGL partout
Force layout 10M nœuds	WebGPU Compute WGSL	×500 vs D3 CPU	Chrome/Safari 2026
D3 layout + Three render	Hybrid CPU layout + GPU render	Meilleur des deux	Oui — pattern standard

